

|                                               |                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework para el Desarrollo de Aplicaciones  | <b>JMX. La guía perdida. Parte 3. JMX, Weblogic y Multitenant.</b><br><i>Por Isaac Ruiz.</i><br>Publicado en Marzo 2017                                                                                                                                                                                |
| Application Express                           | <b>Alcance.</b>                                                                                                                                                                                                                                                                                        |
| Business Intelligence                         | El presente documento detalla cómo conectarse utilizando código java a un servidor de MBeanshospedado en un Weblogic Server 12.2.1. y que contiene un dominio Multitenant.                                                                                                                             |
| Cloud Computing                               | Muestra, como se ejecuta una operación de un MBean utilizando las APIs estándar. Adicionalmente detalla un poco en la creación de un dominio multitenant y repasa algunos conceptos para entender la importancia de esta nueva característica en WLS 12.2.1.                                           |
| Communications                                | Este documento es la 3ª parte de una serie de artículos sobre JMX, puedes consultar los anteriores aquí:                                                                                                                                                                                               |
| Rendimiento y Disponibilidad de Base de datos | <a href="#">Parte 1. JMX, Java Management Extensions. La guía perdida. Los fundamentos.</a><br><a href="#">Parte 2. JMX. La guía perdida. Weblogic y sus MBeans Servers.</a><br><a href="#">Parte 4. JMX. La guía perdida. Accediendo de manera remota a un proceso java.</a>                          |
| Data Warehousing                              | <b>Requerimientos.</b>                                                                                                                                                                                                                                                                                 |
| .NET                                          | El presente documento asume que se tiene cierta experiencia construyendo aplicaciones usando el lenguaje de programación java, asume también que el lector tiene experiencia como desarrollador de aplicaciones y conoce en ese nivel un sistema operativo, por ende,sabe ejecutar tareas a nivel CLI. |
| Lenguajes de Programación Dinámicos           | El documento también asume que se tiene experiencia creando dominios con WLS en alguna de sus versiones más recientes                                                                                                                                                                                  |
| Embedded                                      | Si quieres ver información sobre cómo crear dominios puedes revisar alguno de estos artículos:                                                                                                                                                                                                         |
| Arquitectura Enterprise                       | Weblogic 12.2.1 Instalación y creación de Dominio en Windows 10.<br><a href="https://community.oracle.com/blogs/rugi/2016/05/11/weblogic-1221-instalaci%C3%B3n-en-windows-10">https://community.oracle.com/blogs/rugi/2016/05/11/weblogic-1221-instalaci%C3%B3n-en-windows-10</a>                      |
| Enterprise Management                         | Guía de instalación para Oracle SOA Suite 12c.<br><a href="http://www.oracle.com/technetwork/es/articles/soa/instalacion-oracle-soa-suite-12c-2360582-esa.html">http://www.oracle.com/technetwork/es/articles/soa/instalacion-oracle-soa-suite-12c-2360582-esa.html</a>                                |
| Grid Computing                                | Instalación de Oracle SOA Suite 12c sobre Exalogic<br><a href="http://www.oracle.com/technetwork/es/articles/soa/instalacion-soa-suite-12c-exalogic-2548201-esa.html">http://www.oracle.com/technetwork/es/articles/soa/instalacion-soa-suite-12c-exalogic-2548201-esa.html</a>                        |
| Identidad y Seguridad                         |                                                                                                                                                                                                                                                                                                        |
| Java                                          |                                                                                                                                                                                                                                                                                                        |
| Linux                                         |                                                                                                                                                                                                                                                                                                        |
| Service-Oriented Architecture                 |                                                                                                                                                                                                                                                                                                        |
| SQL & PL/SQL                                  |                                                                                                                                                                                                                                                                                                        |
| Virtualización                                |                                                                                                                                                                                                                                                                                                        |

En este documento, realizaremos la creación de un dominio Multitenant sobre un cluster dinámico, por lo que se recomienda tener un equipo de cómputo con al menos 8GB de memoria RAM.

El ejercicio mostrado se realiza sobre el sistema operativo Windows 10, pero, las instrucciones son aplicables a algún otro sistema operativo soportado por WLS.

**Introducción.**

En esta tercera parte de la serie, continuaremos con el enfoque práctico y seguiremos usando los servidores de MBeans que el WLS 12.2.1 tiene disponibles.

También agregaremos un tema importante que aparece desde la versión 12.2.1 y que encaja perfectamente en esta tendencia de tener procesos aislados y auto-contenidos, me refiero a: multitenant.

Veremos cómo ejecutar una operación de un MBean en alguno de los servers utilizando MBeans nuevos, creados justamente para el tema de multitenant..

En parte el código y las referencias expuestas en este documento son una adaptación del documento:

"Oracle® Fusion Middleware Developing Custom Management Utilities Using JMX for Oracle WebLogic Server 12.2.1".<https://docs.oracle.com/middleware/1221/wls/JMXCU/title.htm>

**Multitenant.**

Empecemos con algunos conceptos antes de arrancar con el tema.

**Contenedores.**

No podemos abordar el tema de multitenant sin tocar el tema de Contenedores.

La tendencia a usar contenedores va más allá de una simple moda, actualmente casi toda solución busca funcionar sobre Docker o alguna otra opción como CoreOS. Repasemos el porqué de esta importancia.

Usar contenedores da entre otras, estas ventajas.

Aislamiento.  
Con el aislamiento se evita la propagación de errores ganando con esto estabilidad.  
Portabilidad.  
A raíz del aislamiento, los contenedores permiten el intercambio/substitución de uno de ellos con la misma facilidad que da crear o eliminar.  
Ligereza.  
Dado que los contenedores aprovechan buena parte de los recursos de su tamaño final es mucho más pequeño.  
Optimización de recursos.  
Cada contenedor puede ejecutarse en ambientes donde los recursos son limitados, evitando con esto tener recursos desperdiciados.  
Autosuficiencia.  
Por definición, cada contenedor debe estar formado por todos los componentes que logran hacer una "unidad ejecutable".

En particular, hablando ya de multitenant, se proporcionan otras ventajas adicionales como:

Rebanada de pastel.  
Los tenants representan una unidad de integración End-to-End, es decir, pueden tocar desde la configuración de acceso, pasando por la aplicación web o servicios de mensajería,y terminando el control de acceso o el repositorio de datos.  
Una mejor organización de los distintos ambientes que conforman el pipeline de liberación de un producto de software para una empresa.  
El punto anterior aunado a mejores herramientas de desarrollo puede propiciar dos cosas más:

Agilidad  
Mejorar el time to market.  
Son el primer paso para que una organización pueda encaminarse a una cultura de SaaS

### Multitenant en WLS

Vamos a realizar el siguiente ejercicio para entender un poco el concepto de multitenant en WLS.

Pensemos en una aplicación WEB estándar; requerimos el WAR, un dataSource, un proveedor JNDI, configurar JMS y listo.

Al momento de desplegarla lo hacemos sobre un dominio (independientemente si está en cluster o no, se trata de sólo 1 dominio).

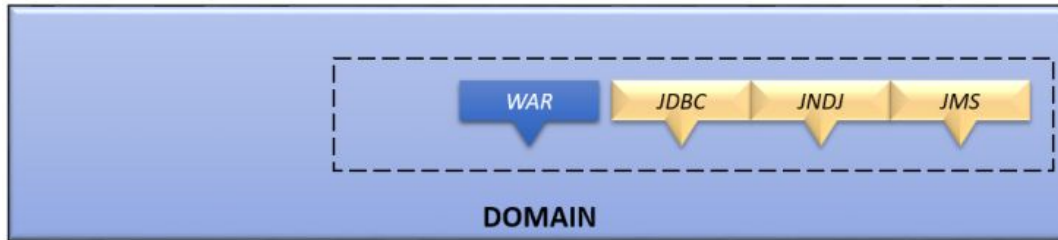


Figura 1. Componentes de una aplicación web típica.

Imaginemos ahora que, nos ha quedado tan optimizada la configuración que ahora nos piden replicarla, lo primero que advertimos mientras hacemos esto en el mismo dominio es que, el 2º deploy del WAR deberá tener otro contexto, ya que no es posible tener dos aplicaciones con el mismo contexto en el mismo dominio.

Algo similar ocurrirá con los recursos, una solución muy recurrida es crear otro dominio, y comenzar a replicar todos los componentes, con esto podremos conservar los nombres.

Así pues, creamos rápidamente un nuevo dominio y replicamos la configuración.

Hemos cumplido con lo solicitado y, una semana después, nos vuelven a pedir replicar la misma configuración.

Esta 2ª ocasión, nos ha llevado un poco menos de tiempo y tenemos ya 3 dominios listos.

Figura 2. Nuestro dominio replicado 2 veces.

El escenario se empieza a complicar cuando nos damos cuenta que el equipo de desarrollo ha decidido por un lado que, es necesario tener esa configuración replicada 4 o más veces y por otro, que será necesario mantener idénticas todas las configuraciones.

Multitenant puede resolver, entre otros, este tipo de escenarios, para ello, se apoya en algunas nuevas piezas que aparecen justamente en la versión 12.2.1 de WLS.

El hecho de replicar varios dominios puede generar cierta complejidad en la administración. Por ello, con multitenant se conserva un solo dominio.

Para lograr esto, lo primero que aparece en el ecosistema de multitenant es algo llamado: Domain Partition, o partición de dominio, y es, como su nombre lo indica un "fragmento" de un dominio, con esto podemos tener el dominio en varias partes, cada una de ellas independiente del resto.

Figura 3. Las particiones de dominio son la primera pieza para el ecosistema multitenant.

Ahora, la siguiente pieza son los ResourceGroups, o grupos de recursos, en esta unidad lógica se agrupan los recursos que en su conjunto forman una "unidad de ejecución replicable", en otras palabras, aquí va todo lo necesario para que un WAR o EAR funcione. ¿Esto último te suena conocido?, es posible, pues es la filosofía de los microservicios.

Por último ¿Cómo resolver el tema del contexto? ¿Cómo evitamos mantener el contexto igual si en un dominio no es posible repetir los contextos de una aplicación?

La respuesta es la última pieza de la configuración de un dominio multitenant.

Los Virtual Targets, o destinos virtuales, sirven precisamente para definir los puntos de acceso. Los VT tienen también una tarea especial, la cual es integrarse con otros componentes de la Fusion Middleware, en particular con el Oracle Traffic Director (OTD).

Todos estos componentes: Virtual Target, ResourceGroups y Domain Partition en su conjunto hacen que pueda existir un dominio multitenant.

Figura 4. Componentes de un dominio multitenant en WLS.

Realmente esta explicación sobre la capacidad multitenant de WLS es muy resumida y limitada, hay un par de videos que te pueden ayudar a ampliar estos conceptos:

WebLogic Multi-Tenancy Fundamentals.  
<https://www.youtube.com/watch?v=cNArBDJlxoc>  
Intro to Multitenancy in WebLogic Server 12.2.1  
[https://www.youtube.com/watch?v=C5GP\\_JB88VY](https://www.youtube.com/watch?v=C5GP_JB88VY)

Ahora que ya conocemos un poco más sobre multitenants, crearemos un dominio con esta característica.

Lo primero que necesitamos es crear un dominio que nos permita aprovechar esta característica del multitenant, para ello existe un template de dominio especial. Teniendo el dominio, comienza la configuración propia de un dominio multitenant; creación de particiones, creación de grupos de

recursos y creación de destinos virtuales.

Creación de dominio.

Comencemos pues con la creación del dominio, para ello únicamente requerimos ejecutar el comando:

```
config.sh
Ubicadon:
ORACLE_HOME/oracle_common/common/bin
PS C:\Oracle\Middleware\Oracle_Home\oracle_common\common\bin> ls
```

Directorio: C:\Oracle\Middleware\Oracle\_Home\oracle\_common\common\bin

| Mode   | LastWriteTime |             | Length | Name                      |
|--------|---------------|-------------|--------|---------------------------|
| ----   | -----         | -----       | -----  | ----                      |
| -a---- | 20/05/2015    | 10:40 a. m. | 986    | clonedunpack.cmd          |
| -a---- | 20/05/2015    | 10:40 a. m. | 1955   | clonedunpack.sh           |
| -a---- | 24/04/2016    | 09:55 p. m. | 8627   | commBaseEnv.cmd           |
| -a---- | 24/04/2016    | 09:55 p. m. | 22787  | commBaseEnv.sh            |
| -a---- | 03/12/2014    | 10:16 a. m. | 739    | commEnv.cmd               |
| -a---- | 31/03/2015    | 11:23 p. m. | 610    | commEnv.sh                |
| -a---- | 31/03/2015    | 11:23 p. m. | 3147   | commExtEnv.cmd            |
| -a---- | 31/03/2015    | 11:23 p. m. | 4129   | commExtEnv.sh             |
| -a---- | 20/05/2015    | 10:40 a. m. | 1259   | config.cmd                |
| -a---- | 20/05/2015    | 10:40 a. m. | 2067   | config.sh                 |
| -a---- | 22/07/2014    | 11:08 a. m. | 902    | configWallet.cmd          |
| -a---- | 12/03/2015    | 11:14 a. m. | 987    | configWallet.sh           |
| -a---- | 12/03/2015    | 11:14 a. m. | 1035   | config_builder.cmd        |
| -a---- | 12/03/2015    | 11:14 a. m. | 1927   | config_builder.sh         |
| -a---- | 24/10/2011    | 03:11 p. m. | 567    | getproperty.cmd           |
| -a---- | 18/06/2013    | 11:04 a. m. | 494    | getproperty.sh            |
| -a---- | 20/05/2015    | 10:40 a. m. | 943    | pack.cmd                  |
| -a---- | 20/05/2015    | 10:40 a. m. | 1945   | pack.sh                   |
| -a---- | 20/05/2015    | 10:40 a. m. | 964    | prepareCustomProvider.cmd |
| -a---- | 20/05/2015    | 10:40 a. m. | 907    | prepareCustomProvider.sh  |
| -a---- | 11/10/2015    | 01:05 a. m. | 1578   | printJarVersions.sh       |
| -a---- | 24/02/2014    | 11:06 p. m. | 613    | qs_config.cmd             |
| -a---- | 24/02/2014    | 11:06 p. m. | 825    | qs_config.sh              |
| -a---- | 20/05/2015    | 10:40 a. m. | 1299   | reconfig.cmd              |
| -a---- | 15/07/2015    | 03:04 p. m. | 2092   | reconfig.sh               |
| -a---- | 24/04/2016    | 09:55 p. m. | 1002   | setHomeDirs.cmd           |
| -a---- | 24/04/2016    | 09:55 p. m. | 731    | setHomeDirs.sh            |
| -a---- | 04/06/2012    | 04:37 a. m. | 1597   | shortenPaths.cmd          |
| -a---- | 20/05/2015    | 10:40 a. m. | 948    | unpack.cmd                |
| -a---- | 20/05/2015    | 10:40 a. m. | 1911   | unpack.sh                 |
| -a---- | 20/05/2015    | 10:40 a. m. | 1497   | wlst.cmd                  |
| -a---- | 15/07/2015    | 03:04 p. m. | 1744   | wlst.sh                   |

Si aún no tienes instalada la infraestructura para tener un ORACLE\_HOME, puede revisar este otro documento:

Weblogic 12.2.1 Instalación y creación de Dominio en Windows 10.  
<https://community.oracle.com/blogs/rugi/2016/05/11/weblogic-1221-instalaci%C3%B3n-en-windows-10> (Hasta el paso 10).

La primera ventana después de ejecutar el config nos permite indicar la ubicación del dominio, toma en cuenta que el nombre de la carpeta final también indica el nombre del dominio, para nuestro ejemplo nuestro dominio se llamará:

otn\_multitenant

Figura 5. Estableciendo el nombre del dominio.

Esta es el paso más importante para la creación de nuestro dominio, debemos seleccionar el template/plantilla que nos permite trabajar con multitenant en WLS, esta plantilla es:

Oracle Enterprise Manager-Restricted JRF - 12.2.1 [em]

Toma en cuenta que al seleccionar esta plantilla se seleccionan automáticamente otras plantillas que se requieren como dependencias.

Figura 6. Seleccionando el template para multitenant.

A partir de aquí la mayoría de las opciones que nos ofrezca el asistente, serán por omisión.

Esto incluye, la ruta para colocar las aplicaciones.

Figura 7. Ruta de las aplicaciones del dominio.

La siguiente pantalla nos solicita el usuario y passwords para el usuario admin, asegúrate de dejar el password en un lugar seguro.

Para este ejemplo usaremos:

|         |          |
|---------|----------|
| Usuario | weblogic |
|         |          |

|                 |          |
|-----------------|----------|
| <b>Password</b> | welcome1 |
|-----------------|----------|

Figura 8. Definición de usuario y contraseña para el administrador.

Nuestro dominio estará en modo de desarrollo y usaremos el JDK oficial.

Figura 9. Modo del dominio y elección del JDK.

No haremos configuraciones especiales en este momento, solo requerimos el servidor de administración con sus opciones por default.

Figura 10. Configuración extra.

Antes de terminar, nos aparece el ya conocido resumen.

Figura 11. Resumen de configuración

Se comienza la creación del dominio.

Figura 12. Creación del dominio.

Nuestro dominio está listo para ser iniciado.

Figura 13. Resumen final del dominio creado.

Si arrancamos el dominio:

```
C:\Oracle\Middleware\Oracle_Home\user_projects\domains\otn_multitenant\bin\ .\startWebLogic.cmd
```

Gracias al template que elegimos, podemos ver la consola del Enterprise Manager.

<http://localhost:7001/em>

Figura 14. Pantalla de acceso al Enterprise manager.

La consola de weblogic sigue accesible.

<http://localhost:7001/console>

Figura 15. Pantalla de acceso a la consola del WLS.

Vamos a hacer una pequeña configuración para continuar. Seleccionaremos nuestro admin server.

Dominio->Servidores->AdminServer

Figura 16. Listado de servidores.

Y nos aseguraremos de que la dirección de recepción está adecuadamente establecida, si el campo te aparece en blanco, asígnale (le asignaremos localhost porque todo el ejercicio se realiza sobre una sola maquina). Hacemos click en grabar y nos debe de aparecer un mensaje indicando que la operación ha sido ejecutada correctamente.

Figura 17. Modificación de

Debemos de reiniciar el admin server para que el cambio tenga efecto. Una vez reiniciado podemos continuar.

¿Recuerdas el código del artículo anterior para visualizar los MBeans del dominio? Es el ejercicio 1.

Revisa el 2ª parte de esta serie y ejecuta la clase del Ejercicio 1. El código te permite visualizar los MBeans de un servidor.

Ejecútalo nuevamente, para revisar los MBeans de nuestro dominio recién creado.

```
com.bea:Name=oracle.bi.adf.view.slib#1.0@12.2.1.0.0,Location=otn_multitenant,Type=Library

com.bea:Location=AdminServer,WseeClientConfigurationRuntime=oracle.sysman.emInternalSDK.sdkas.general.pojo.fwk.swb.ruei.SesdiagSoap12HttpClient/sesdiagService,WebAppComponentRuntime=AdminServer_/em,ServerRuntime=AdminServer,ApplicationRuntime=em,Name=getAlertInfo,WseePortConfigurationRuntime=SesdiagSoap12HttpPort,Type=WseeOperationConfigurationRuntime

com.bea:Name=OHW_it,ServerRuntime=AdminServer,Location=AdminServer,Type=ServletRuntime,ApplicationRuntime=em,WebAppComponentRuntime=AdminServer_/em

com.bea:Name=emagentsdkimplpriv_jar#12.4@12.1.0.4.0,Type=Library

com.oracle:type=OVD,context=ids,name=AdaptersConfig

com.bea:Name=adf.oracle.domain.webapp#1.0@12.2.1.0.0,Type=LibDeploymentRuntime
```

Puedes ver el listado complete aquí

<https://gist.github.com/rugi/53fbfc7bc54836bec1c3649bc95a990a>

Son 1058 MBeans, toma en cuenta este número, lo compararemos con otro al final de este documento.

**Creación del tenant**  
**Objetivo.**

Ahora, debemos definir sobre el dominio recién creado una partición, un resourcegroup y un virtual target.

**Creación del tenant.**

Para este ejercicio definiremos un cluster dinámico, sobre el irá desplegado nuestro dominio Multitenant.

Aún y cuando nuestro cluster estará ubicado en la misma maquina requerimos echar a andar el NodeManager ya que es el mecanismo por medio del cual el AdminServer envía las ordenes al resto de los servidores.

**Levantar weblogic.**

Dentro de nuestro dominio levantamos nuestro WLS

```
ORACLE_HOME\user_projects\domains\otn_multitenant\bin>.\startWebLogic.cmd
```

**Levantar nodeManager.**

Y nuestro node manager.

```
ORACLE_HOME\user_projects\domains\otn_multitenant\bin>.\startNodeManager.cmd
```

Con esto tendremos 2 ventajas en ejecución.

Figura 18. Ventanas de Windows arrancando el dominio recién creado.

Una vez que los logs indiquen que el server está en RUNNING podemos entrar a la consola del EM.

**Creación de la maquina.**

A partir de este momento, utilizaremos únicamente el Enterprise Manager, accederemos a la pantalla principal con la URL: <http://localhost:7001/em>

Figura 19. Pantalla de acceso al Enterprise Manager.

Lo primero que debemos de crear es una , requerimos esto pues, nuestra configuración de multitenant descansará sobre un cluster.

Llegamos vía:

Dominio->Entorno->Máquinas

Figura 20. Creación de una nueva 1.

Hacemos click en [Crear]

Figura 21. Creación de una nueva máquina 2.

Aparece ahora el asistente para crear una máquina, si no estás muy familiarizado con el Enterprise manager (EM), los asistentes incluyen en la parte superior un indicador de avance, para que siempre tengamos claro cuantas pantallas nos hacen falta para terminar una tarea.

Debemos dar de entrada dos valores:

|                         |            |
|-------------------------|------------|
| Nombre (de la maquina): | maquina-01 |
| Machine OS:             | Other      |

En la opción de sólo tenemos dos opciones, dado que nuestro ejemplo está siendo sobre Windows, elegimos .

Figura 22. Estableciendo nombre y tipo de Sistema Operativo.

Damos click en [Siguiente].

La siguiente pantalla nos permite cambiar los valores para el , dejaremos los valores por default.

Figura 23. Propiedades del gestor de nodos.

Por último, obtenemos un resumen antes de hacer click en el botón [Crear]

Figura 24. Resumen de la maquina a crear.

Nos aparece un mensaje indicando el éxito de la operación.

Figura 25. Mensaje de operación satisfactoria.

Tenemos ya definida la maquina-01.

#### **Creación del cluster.**

Vamos ahora a continuar con la creación del cluster.

Dominio->Entorno->Clusters

Figura 26. Menú. Creación de nuevo cluster.

Para este ejercicio, crearemos un , un cluster dinámico tiene la característica de poder aumentar su tamaño de nodos de manera dinámica.

Se define un rango tamaño máximo de nodos, y un tamaño inicial. Se acopla perfectamente a lo requerido por dominios multitenant.

Figura 27. Creación de cluster dinámico.

Una vez elegido el cluster dinámico, Dejaremos los valores por default.

Toma en cuenta que el nombre sugerido para el cluster es Cluster-01

Figura 28. Definición del nombre del cluster.

Dejaremos también los valores por default para los tamaños inicial y máximo del cluster, el prefijo para cada servidor se genera a partir del nombre del cluster, pero puede ser modificado sin problemas. Continuamos.

Figura 29. Dimencionamiento del cluster.

En la siguiente pantalla se establece la relación con la o las máquinas que soportarán al cluster, elegiremos la opción de una sola máquina y seleccionaremos la que recién hemos creado.

Hacemos click en [Siguiente].

Figura 30. Enlace de máquinas para el cluster.

La siguiente pantalla nos muestra ahora los puertos, dejaremos los valores por default.

Hacemos click en [Siguiente].

Figura 31. Configuración de puertos para el clúster.

Por último, nos aparece el resumen y el botón de [Crear].

Figura 32. Resumen de características del cluster a crear.

Después de hacer click en el botón [crear] nos aparece un mensaje indicando que la operación se ha realizado satisfactoriamente.

Figura 33. Mensaje indicando creación exitosa.

Nuestro cluster aparece ya en la lista de clusters disponibles, ahora lo vamos a iniciar, esto es, iniciar todos los servidores que lo conforman. Toma en cuenta que esta operación puede consumir memoria RAM, por lo que, asegúrate de tener al menos 8GB de RAM.

Para seleccionar nuestro cluster, basta con hacer click en el cuadro con que inicia el renglón de nuestro cluster. Eso activa los menus superiores y ahora podemos hacer click en:

Control-> Iniciar.

Figura 34. Iniciando el cluster.

Después de unos minutos, el cluster se encuentra funcionando correctamente. Puedes refrescar la página de manera normal (F5) o usar el icono de refresh que ya conocemos.

Figura 35. Indicador de cluster en funcionamiento.

Nuestro cluster está operando correctamente.

Hasta este punto llevamos configurado lo siguiente:

Figura 36. Relación entre el cluster-01 y la maquina-01.

**Creación del Virtual Target.**

Ahora, debemos definir un Virtual Target.

Para crear el Virtual target, accedemos al menú:

Domain->Entorno->Destinos Virtuales.

Figura 37. Creación de TargetVirtual/DestinoVirtual

Elegimos [Crear].

Figura 38. Creación de TargetVirtual/DestinoVirtual.

Usaremp os los siguientes valores:

|                 |           |
|-----------------|-----------|
| Nombre          | VT-01     |
| Prefijo de URI  | /dp01     |
| Nombre del host | localhost |

Figura 39. Definición de nombre, prefijo y host del TargetVirtual/DestinoVirtual.

La siguiente pantalla nos solicita indiquemos sobre que servidor o cluster se creará el Virtual Target. Elegiremos nuestro cluster:

Cluster-01.

Figura 40. Selección de servidor o cluster.

Nuestro virtual target aparece ya en el listado.

Figura 41. VirtualTarget/DestinoVirtual creado correctamente.

Nuestro virtual target está listo para ser utilizado.  
Así va nuestra configuración:

Figura 42. VirtualTarget dentro de nuestro dominio.

Vamos ahora a crear la partición de dominio.  
**Creación del DomainPartition.**

Para crear una partición, debemos acceder a la opción de menú.  
Dominio->Entorno->Particiones de Dominio.

Figura 43. Crear partición de dominio.

La creación de particiones requiere activar cierta configuración que nos obliga a reiniciar el AdminServer, sólo será necesario reiniciar este server y no todo el cluster.  
La configuración que debemos activar se realiza haciendo click en el botón [EnableLifecycle Manager].

Figura 44. Habilitar

Hacer esto nos mostrará un mensaje indicando que debemos reiniciar el AdminServer.

Figura 45. Solicitud de reinicio de servidores.

En este punto es preciso que se detenga el proceso del AdminServer, lo tenemos en una de las dos ventanas, puedes detenerlo con la combinación de teclas CTRL+C.  
Una vez que se detenga completamente, hay que volver a iniciarlo, una vez reiniciado volveremos a firmarnos en el enterprise manager y regresamos a la pantalla donde pausamos.  
Dominio->Entorno->Particiones de Dominio.  
Ya podemos hacer click en el botón [Crear].

Figura 46. Crear particiones de dominio.

Usaremos los siguientes valores para la partición que vamos a crear:

|                       |         |
|-----------------------|---------|
| Nombre:               | dp1     |
| Dominio de seguridad: | Ninguno |

Figura 47. Definiendo nombre de la partición.

Hacemos click en [Siguiente]  
Ahora debemos elegir que Virtual Target vamos a utilizar y cuál será el que será usado por default.

Figura 48. Estableciendo destinos para la partición.

Por último, definiremos el nombre del resourcesGroup/grupo de recursos de esta partición, usaremos los siguientes valores:

|                                |         |
|--------------------------------|---------|
| Nombre del Grupo de recursos   | RG01    |
| Plantilla de Grupo de recursos | Ninguno |
| Destinos seleccionados         | VT-01   |

Figura 49. Creando Grupo de Recursos inicial.

Hacemos click en [Siguiente].



Vemos nuevamente un resumen antes de dar click en [Crear]

Figura 50. Resumen de la partición a crear.

Nuestra partición llamada dp1 ya está definida, vamos a iniciarla.

Para ello la seleccionamos y activamos con la opción:

Control->Iniciar.

Figura 51. Arrancando partición.

Se abrirá un cuadro de diálogo indicando que la operación de inicio de nuestra partición se encuentra ejecutándose.

Figura 52. Mensajes indicando el arranque de la partición.

Una vez concluida la operación, nuestra partición debe estar ya ejecutándose.

Figura 53. Partición en funcionamiento.

Listo, nuestra partición de dominio está funcionando.

Así ha quedado nuestra configuración:

Figura 54. Dominio Multitenant básico.

¿Y ahora? ¿Qué sigue? Dejaremos hasta ese punto a nuestro dominio, con una partición funcionando sobre un cluster.

Ahora, estamos listos para conocer cuáles son los nuevos métodos Mbeans que vienen con esta versión de WLS para trabajar con multitenant.

#### Revisión de MBeans.

Volvamos a revisar el listado de MBeans.

Veremos MBeans como los siguientes:

```
com.bea:Name=VT-01,Type=WebServer,VirtualTarget=VT-01
```

```
com.bea:Name=ConnectorService,ServerRuntime=Cluster-01-1,Location=Cluster-01-1,Type=ConnectorServiceRuntime
,PartitionRuntime=dp1
```

```
com.bea:Name=dp1,Location=otn_multitenant,Type=SelfTuning,Partition=dp1
```

```
com.bea:Name=RG01,Location=otn_multitenant,Type=ResourceGroup,Partition=dp1
```

```
com.bea:Name=Cluster-01-Template,Type=FederationServices,ServerTemplate=Cluster-01-Template
```

Estos MBeans ya referencian tanto a la partición como al target virtual.

Si volvemos a ejecutar el Ejercicio 1 de la entrega anterior podremos ver cuantos MBeans son los que ahora existen. Puedes ver el listado completo de MBeans aquí:

<https://gist.github.com/rugi/2f1ba66482323a53616f6e3af6cc4dd5>

Los MBeans ahora son en total 1987, toma en cuenta que aún no desplegamos algo en el dominio; realmente los servidores de MBeans de WLS proporcionan MBeans para administrar prácticamente cualquier parte del dominio.

#### Ejecución de operaciones.

Tenemos ya un dominio con un tenant listo para utilizarse, continuaremos ahora revisando la API para la versión 12.2.1 de WLS.

Nuestro objetivo en esta ocasión será el equivalente de lo que hicimos anteriormente con un dominio, sólo que, en lugar de preguntar por el estado de los servidores, preguntaremos por el estado de cada partición.

Realmente las clases dan lugar a poder detener incluso un resourcegroup, pero, para fines ilustrativos llegaremos sólo a nivel de partición.

#### Accediendo al MBean Server del dominio.

Para este ejercicio, nuevamente accedemos al servidor de dominio de nuestro WLS.

Accederemos directamente al: **DomainRuntimeService**

EsteMBean tiene su definición en la clase:

```
weblogic.management.mbeanservers.domainruntime.DomainRuntimeServiceMBean
```

Puedes ver la documentación completa en:

DomainRuntimeService  
<https://docs.oracle.com/middleware/1221/wls/WLAPI/weblogic/management/mbeanservers/domainruntime/DomainRuntimeServiceMBean.html>

En la ocasión anterior, no ejecutamos método alguno de la clase, solamente accedimos directamente a la propiedad: ServerRuntimes.

Figura 55. Composición parcial de la clase DomainRuntimeServiceMBean.

En esta ocasión debemos invocar un método, ya que no existe una propiedad que nos devuelva la lista completa de particiones. Revisando la documentación, vemos que nos puede servir el método:

findPartitionRuntimes

|                                 |                                                                                                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------|
| abstractPartitionRuntimeMBean[] | findPartitionRuntimes(StringpartitionName)                                                                     |
|                                 | Returns all current PartitionRuntimeMBean instances for given partition on all targeted servers in the domain. |

Figura 56. Relación entre el método findPartitionRuntimes y un arreglo de PartitionRuntimeMBean.

Teniendo el PartitionRuntimeMBean ya podremos conocer su estado, y ejecutar alguna operación.

PartitionRuntimeMBean  
<https://docs.oracle.com/middleware/1221/wls/WLAPI/weblogic/management/runtime/PartitionRuntimeMBean.html>

Nos interesa conocer estos 3 métodos:

| Método        | Descripción                                                |
|---------------|------------------------------------------------------------|
| getName       | Recupera el nombre de esta partición.                      |
| getState      | Recupera el estado actual de la partición.                 |
| getServerName | Recupera el nombre del servidor asociado a esta partición. |
| suspend       | Suspende la partición.                                     |
| resume        | Reanuda la partición actual.                               |

Teniendo claro los métodos que requerimos de cada MBean podemos comenzar a codificar.

Invoke

En el artículo pasado conocimos al método **getAttribute**, para recuperar valores de atributos de un MBean, en esta ocasión conoceremos el método: **invoke**.

El método tiene la siguiente firma:

```
Object invoke(ObjectName name, String operationName, Object[] params,String[] signature)
    throws InstanceNotFoundException, MBeanException, ReflectionException, IOException
```

El método recibe, un ObjectName, el nombre del método a ejecutar, un arreglo de objetos que representan los parámetros que recibe el método, y al final un arreglo de Strings que representan las clases que corresponden a cada uno de los parámetros.

El método devuelve un Objeto, debemos recordar que, esto es así para que después hagamos el cast a algún otro tipo de respuesta, un arreglo, por ejemplo.

Veamos como ejemplo el métodofindPartitionRuntimes , el cual nos va servir para recuperar el listado de PartitionRuntimeMBean.

El método sólo recibe un parámetro, un String que representa el nombre de la partición a buscar, nosotros queremos ejecutar:

```
findPartitionRuntimes("dp1");
```

Ya que "dp1" es el nombre de nuestra partición.

La línea anterior se convierte en:

```
public static ObjectName[] getPartitionsRuntimes() throws Exception {
    Object opParams[] = {"dp1"};
    String opSig[] = {String.class.getName(),};
    return (ObjectName[])
connection.invoke(service,
"findPartitionRuntimes", opParams, opSig);
}
```

opParams lleva solo un elemento, nuestro String con el valor de "dp1"  
opSigde igual manera, solo está formado por un elemento, el cual corresponde al tipo de dato de nuestro parámetro: String (java.lang.String).

¿Y si el método no recibe parámetros? Lo veremos a continuación con los métodos: suspend() y resume()

Estado de una partición.

Para conocer el estado de la partición solamente requerimos preguntar por el valor de los atributos: Name, State y NameServer.

Es algo ya conocido, similar a lo que hicimos con los servers en el capítulo anterior de esta serie.

```
public void printDomainPartitionState() throws Exception {
    ObjectName[] partitionT = getPartitionsRuntimes();
    System.out.println
    ("----- Partition state -----");
    int length = (int) partitionT.length;
    for (inti = 0; i< length; i++) {
        String name = (String) connection.getAttribute(partitionT[i], "Name");
        String state = (String) connection.getAttribute(partitionT[i], "State");
        String serverName =
        (String) connection.getAttribute(partitionT[i], "ServerName");
        System.out.println("Partition name: " +
        name + ". Partition state: " +
        state + ". Server Name:" + serverName);
    }//for
    System.out.println("-----          #####          -----");
} //method
```

El método que encapsula estose llama:

```
printDomainPartitionState();
```

Cuando mandemos a invocar a ese método debemos tener una salida como la siguiente:

```
----- Partition state -----
Partition name: dp1. Partition state: RUNNING. Server Name:Cluster01-1
Partition name: dp1. Partition state: RUNNING. Server Name:Cluster01-2
-----          #####          -----
```

### Suspender una partición.

Para la suspensión de la partición invocaremos al método suspend, este método no requiere parámetros por lo que, como imaginábamos, solo es necesario pasarle null al método invoke en los argumentos correspondientes.

```
public void suspendDomainPartition() throws Exception {
    ObjectName[] partitionT = getPartitionsRuntimes();
    int length = (int) partitionT.length;

    for (inti = 0; i< length; i++) {
        String name =
        (String) connection.getAttribute(partitionT[i], "Name");
        String serverName =
        (String) connection.getAttribute(partitionT[i], "ServerName");
        Object responseForceShutdown =
        connection.invoke(partitionT[i], "suspend", null, null);
        System.out.println
        ("Se envió mensaje para suspender la particion:" +
        name + " en el server:" + serverName);
    }//for
} //method
```

El método que encapsula esto se llama:

```
suspendDomainPartition();
```

Cuando mandemos a invocar a ese método debemos tener una salida como la siguiente:

```
Se envió mensaje para suspender la particion:dp1 en el server:Cluster01-1
Se envió mensaje para suspender la particion:dp1 en el server:Cluster01-2
```

Podremos volver a preguntar por el estado

```
printDomainPartitionState();
```

Y ahora veremos

```
----- Partition state -----
Partition name: dp1. Partition state: ADMIN. Server Name:Cluster01-1
Partition name: dp1. Partition state: ADMIN. Server Name:Cluster01-2
-----          #####          -----
```

Si vamos al enterprise manager a la sección de particiones, deberemos ver algo como esto:

Figura 57. Partición en estado de ADMIN.

### Reactivar una partición.

Por último, resume es un método que tampoco recibe parámetros por lo que su invocación con invoke es idéntica a suspend.

```
public void resumeDomainPartition() throws Exception {
    ObjectName[] partitionT = getPartitionsRuntimes();
    int length = (int) partitionT.length;

    for (inti = 0; i< length; i++) {
        String name =
        (String) connection.getAttribute(partitionT[i], "Name");
        String serverName =
        (String) connection.getAttribute(partitionT[i], "ServerName");
        Object responseForceShutdown =
        connection.invoke(partitionT[i], "resume", null, null);
    }
```

```

System.out.println("Se envió mensaje para reiniciar la particion:" +
name + " en el server:" + serverName);
    } //for
} //method

```

El método que encapsula esto se llama:

```

resumeDomainPartition();

```

Cuando mandemos a invocar a ese método ahora veremos:

```

Se envió mensaje para reiniciar la particion:dp1 en el server:Cluster01-1
Se envió mensaje para reiniciar la particion:dp1 en el server:Cluster01-2

```

Una vez más podremos llamar al método para conocer el estado:

```

printDomainPartitionState();

```

Y ahora veremos nuevamente cada fragmento de la partición en RUNNING

```

----- Partition state -----
Partition name: dp1.    Partition state: RUNNING. Server Name:Cluster01-1
Partition name: dp1.    Partition state: RUNNING. Server Name:Cluster01-2
-----
          #####
-----

```

Y, el enterprise manager debe de mostrarnos la partición .

Figura 58. Partición funcionando.

A continuación, va el listado completo de la clase para que puedas ejecutarlo localmente, el método main está preparado para recibir un parámetro desde la línea de comandos, esto nos va a ayudar a ejecutar la secuencia que hemos visto.

#### Listado completo.

```

import java.io.IOException;
import java.net.MalformedURLException;
import java.util.Hashtable;
import javax.management.MBeanServerConnection;
import javax.management.MalformedObjectNameException;
import javax.management.ObjectName;
import javax.management.remote.JMXConnector;
import javax.management.remote.JMXConnectorFactory;
import javax.management.remote.JMXServiceURL;
import javax.naming.Context;

/**
 *
 * @author S&P Solutions S.A de C.V.
 */
public class ManageDomainPartition {

    private static MBeanServerConnection connection;
    private static JMXConnector connector;
    private static final ObjectName service;

    // Initializing the object name for DomainRuntimeServiceMBean
    // so it can be used throughout the class.
    static {
        try {
            service = new ObjectName("com.bea:Name=DomainRuntimeService,Type=weblogic.management.
mbeanservers.domainruntime.DomainRuntimeServiceMBean");
        } catch (MalformedObjectNameException e) {
            throw new AssertionError(e.getMessage());
        }
    }

    /**
     * Initialize connection to the Domain Runtime MBean Server
     */
    public static void initConnection(String hostname, String portString,
        String username, String password) throws IOException,
        MalformedURLException {

        String protocol = "t3";
        Integer portInteger = Integer.valueOf(portString);
        int port = portInteger.intValue();
        String jndiroot = "/jndi/";
        String mserver = "weblogic.management.mbeanservers.domainruntime";
        JMXServiceURL serviceURL = new JMXServiceURL(protocol, hostname,
            port, jndiroot + mserver);
        Hashtable environment = new Hashtable();
        environment.put(Context.SECURITY_PRINCIPAL, username);
        environment.put(Context.SECURITY_CREDENTIALS, password);
        environment.put(JMXConnectorFactory.PROTOCOL_PROVIDER_PACKAGES,
            "weblogic.management.remote");
        environment.put("jmx.remote.x.request.waiting.timeout", new Long(10000));
        connector = JMXConnectorFactory.connect(serviceURL, environment);
        connection = connector.getMBeanServerConnection();
        System.out.println("< ConexionEstablecida >");
    }

    public static ObjectName[] getPartitionsRuntimes() throws Exception {

        Object opParams[] = {"dp1"

```

```

    };

    String opSig[] = {String.class.getName(),};
    return (ObjectName[]) connection.invoke(service, "findPartitionRuntimes", opParams, opSig);
}

/*
 * Iterate through ServerRuntimeMBeans and get the name and state
 */
public void printDomainPartitionState() throws Exception {
    ObjectName[] partitionT = getPartitionsRuntimes();
    System.out.println("----- Partition state -----");
    int length = (int) partitionT.length;
    for (inti = 0; i < length; i++) {
        String name = (String) connection.getAttribute(partitionT[i], "Name");
        String state = (String) connection.getAttribute(partitionT[i], "State");
        String serverName = (String) connection.getAttribute(partitionT[i], "ServerName");
        System.out.println("Partition name: " + name + ". Partition state: " + state + ". Server Name: " + serverName);
    }
    System.out.println("----- ##### -----");
}

public void suspendDomainPartition() throws Exception {
    ObjectName[] partitionT = getPartitionsRuntimes();
    int length = (int) partitionT.length;

    for (inti = 0; i < length; i++) {
        String name = (String) connection.getAttribute(partitionT[i], "Name");
        String serverName = (String) connection.getAttribute(partitionT[i], "ServerName");
        Object responseForceShutdown = connection.invoke(partitionT[i], "suspend", null, null);
        System.out.println("Se envió mensaje para suspender la particion:" + name + " en el server:" + serverName);
    }
}

public void resumeDomainPartition() throws Exception {
    ObjectName[] partitionT = getPartitionsRuntimes();
    int length = (int) partitionT.length;

    for (inti = 0; i < length; i++) {
        String name = (String) connection.getAttribute(partitionT[i], "Name");
        String serverName = (String) connection.getAttribute(partitionT[i], "ServerName");
        Object responseForceShutdown = connection.invoke(partitionT[i], "resume", null, null);
        System.out.println("Se envió mensaje para reiniciar la particion:" + name + " en el server:" + serverName);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length == 1) {
        String operacion = args[0].trim().toLowerCase();
        System.out.println("Ejecutando <"+operacion+">");
        String hostname = "localhost";
        String portString = "7001";
        String username = "weblogic";
        String password = "welcome1";
        ManageDomainPartition domainPartition = new ManageDomainPartition();
        initConnection(hostname, portString, username, password);
        if (operacion.equals("state")) {
            domainPartition.printDomainPartitionState();
        }
        if (operacion.equals("suspend")) {
            domainPartition.suspendDomainPartition();
        }
        if (operacion.equals("resume")) {
            domainPartition.resumeDomainPartition();
        }
        connector.close();
        System.out.println("< Conexion cerrada >");
    } else {
        System.out.println(" Use alguno de los siguientes comandos:");
        System.out.println("          state | suspend | resume ");
        System.out.println("Ejemplo:");
        System.out.println("          ManageDomainPartition state");
    }
}
}
}

```

### Compilación.

Para compilar la clase, debemos de agregar estos archivos en la carpeta donde tenemos la clase:

```

ORACLE_HOME\wlserver\server\lib\wljmxclient.jar
ORACLE_HOME\wlserver\server\lib\wlclient.jar

```

Imaginando que tenemos la clase en la carpeta jmx03/

Coloraremos esos 2 jars, más nuestra clase en la misma carpeta.

```

-a---- 09/03/2017 09:33 p. m.      6801 ManageDomainPartition.java
-a---- 24/04/2016 09:54 p. m.      2655081 wlclient.jar
-a---- 24/04/2016 09:55 p. m.      241029 wljmxclient.jar

```

[Y compilamos así:

```
%>javac -cp "wljmxclient.jar;wlclient.jar;." ManageDomainPartition.java
```

Listo, debemos tener nuestro .class en la misma carpeta

Ejecución.

Ahora para ejecutar

```
%>java -cp "wljmxclient.jar;wlcient.jar;." ManageDomainPartition
```

Si no le pasamos argumentos veremos una salida como la siguiente:

```
Use alguno de los siguientes comandos:
state| suspend | resume
Ejemplo:
ManageDomainPartition state
```

Y ahora, con nuestro dominio en funcionamiento podemos seguir la secuencia que acabamos de detallar.

Validamos el estado.

```
%>java -cp "wljmxclient.jar;wlcient.jar;." ManageDomainPartition state
```

Suspendemos la partición.

```
%>java -cp "wljmxclient.jar;wlcient.jar;." ManageDomainPartitionsuspend
```

Validamos el estado.

```
%>java -cp "wljmxclient.jar;wlcient.jar;." ManageDomainPartition state
```

Reactivamos la partición

```
%>java -cp "wljmxclient.jar;wlcient.jar;." ManageDomainPartition resume
```

Y validamos que se encuentre nuevamente en funcionamiento.

```
%>java -cp "wljmxclient.jar;wlcient.jar;." ManageDomainPartition state
```

Listo, hemos detenido y reactivado una partición usando JMX

Alcance de cada servidor.

Hasta el momento sólo nos hemos conectado al servidor:

```
com.bea:Name=DomainRuntimeService,Type=weblogic.management.mbeanservers.domainruntime.DomainRuntimeServiceMBean
```

Pero, ¿Y los otros servidores de MBeans?

Recordemos la tabla que vimos en el post anterior

| MBean Server              | Objetivo                                                                                                                                                                                                                                                                                         |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DomainRuntimeMBean Server | Proporciona acceso a los MBeans de servicios del dominio completo, esto incluye aplicaciones desplegadas, servidores JMS, dataSources, etc. Es el punto de acceso para acceder de manera jerárquica toda la información del dominio, es decir, de todos los servidores que conforman el dominio. |
| RuntimeMBean Server       | Proporciona acceso a MBeansde tiempo de ejecución y puede activarlos dentro de la configuración.                                                                                                                                                                                                 |
| EditMBean Server          | Proporciona el punto de entrada para administrar la Configuración del dominio.                                                                                                                                                                                                                   |

Hemos estado conectándonos al 1er MBeanServer porque nuestra intención hasta el momento ha sido acceder a MBeans que se encuentra a lo largo del dominio, en esta ocasión a una partición que se encuentra asignada a un cluster.

Acceder a los otros servidores representa tener escenarios más avanzados, por ejemplo, entrar el RunTimeMBean Server nos serviría para inspeccionar algúnMBean que nos informen sobre la creación de un nuevo dataSource o de alguna nueva partición.

Cada servidor tiene alcances especiales y por ello existe documentación que describe lo que se puede y no se puede realizar.

Esta documentación se distribuye por versión de WLS y, para la versión 12.2.1 la URL es:

```
The WebLogic Server® MBean Reference.
https://docs.oracle.com/middleware/1221/wls/WLMBR/core/index.html
```

Figura 59. Documentación oficial de los MBean Servers que expone WLS 12.2.1.

Probablemente más adelante dentro de esta serie mostremos algunos escenarios de uso de los otros servidores de MBeans.

Conclusión.

En esta tercera entrega hemos explorado un dominio multitenant, una de las más recientes características de WLS 12c, hemos visto como sigue siendo posible administrar los componentes de cadatenant usando JMX.

Para ello hemos invocado una operación de un MBean, dentro de un dominio multitenant que nosotros mismos hemos creado, realizando todo únicamente a través de las API's estándar.

Ya tenemos pues los conocimientos suficientes para poder comenzar a agregar más herramientas a este ecosistema; en la primera parte de este artículo ya conocimos una JConsole, pero hay otras como VisualVM.

A veces solo queremos monitorear los valores de ciertos MBeans y, no tiene mucho sentido hacerlo con código.

Enlaces.

JSR-000003: JavaTM Management Extensions (JMX) Specification.  
<https://jcp.org/en/jsr/detail?id=3>  
JSR-000003 JavaTM Management Extensions (JMX) (Maintenance Release 4)  
<https://jcp.org/aboutJava/communityprocess/mrel/jsr003/index4.html>  
JSR 000255 - JMX Specification, version 2.0  
<https://jcp.org/en/jsr/detail?id=255>  
JSR 160: Java Management Extensions (JMX) Remote API.  
<https://jcp.org/en/jsr/detail?id=160>  
Oracle® Fusion Middleware Developing Custom Management Utilities Using JMX for Oracle WebLogic Server.  
<https://docs.oracle.com/middleware/1221/wls/JMXCU/title.htm>  
Oracle Fusion Middleware Java API Reference for Oracle WebLogic Server 12c (12.2.1)  
<https://docs.oracle.com/middleware/1221/wls/WLAPI/index.html>  
Repositorio con el código usado en esta serie de artículos.  
<https://github.com/rugijmx>

**Isaac Ruiz Guerra** (@rugi), es programador Java yConsultor TI. Especializado en integración de sistemas, fundamentalmente relacionados con el sector financiero. Actualmente forma parte del equipo de S&P Solutions. Escribe en su blog personal [xhubacubi.blogspot.com](http://xhubacubi.blogspot.com), pero también participa y pertenece a [www.javahispano.org](http://www.javahispano.org) y a [www.javamexico.org](http://www.javamexico.org)

*Este artículo ha sido revisado por el equipo de productos Oracle y se encuentra en cumplimiento de las normas y prácticas para el uso de los productos Oracle.*

E-mail this page      Printer View

Contáctenos

Ventas en EE. UU.:  
+1.800.633.0738

Reciba una llamada de Oracle

Contactos globales

Directorio de soporte

Acerca de Oracle

Información de la empresa

Comunidades

Carreras

Cloud

Descripción general de Cloud Solutions

Software (SaaS)

Plataforma (PaaS)

Infraestructura (IaaS)

Datos (DaaS)

Prueba gratuita de la nube

Eventos

Oracle Code

JavaOne

Todos los eventos de Oracle

Acciones principales

Descargue Java

Descargue Java para desarrolladores

Pruebe Oracle Cloud

Suscríbase a los correos electrónicos

Noticias

Sala de prensa

Revistas

Historias de éxito de los clientes

Blogs

Temas clave

ERP, EPM (Finanzas)

HCM (RR. HH. y talento)

Mercadeo

CX (Ventas, servicio, comercio)

Soluciones sectoriales

Base de datos

MySQL

Middleware

Java

Sistemas de ingeniería